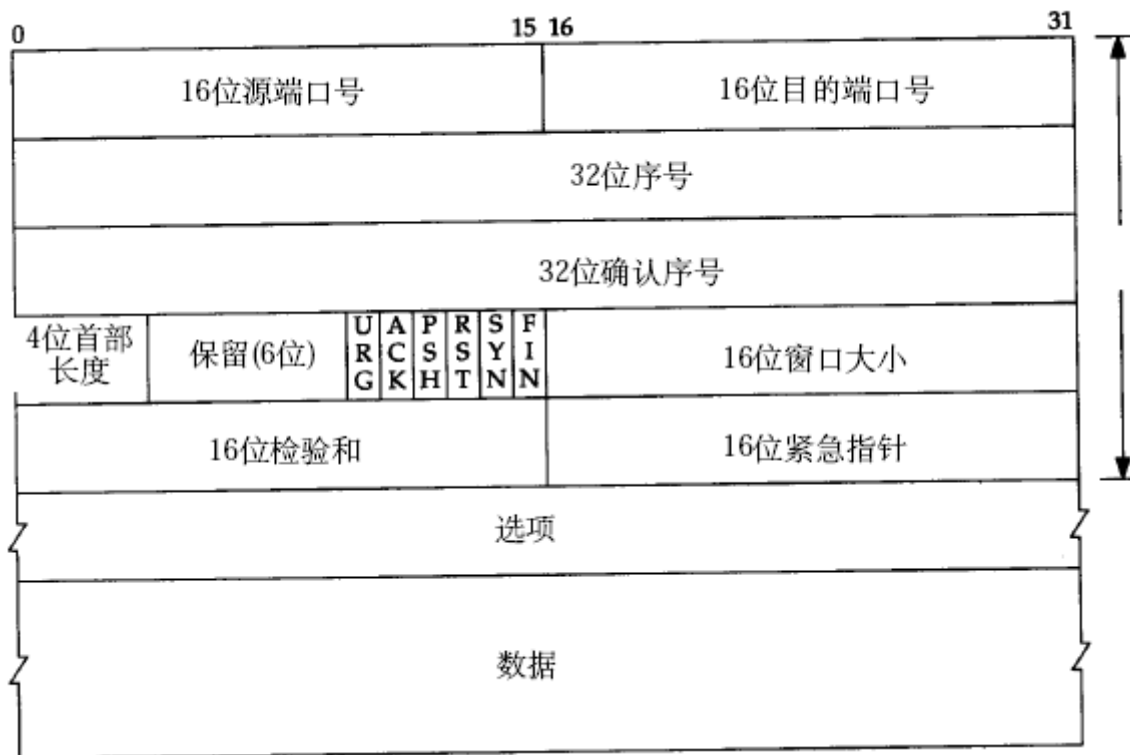


再议TCP协议及socket并发编程深入API解析

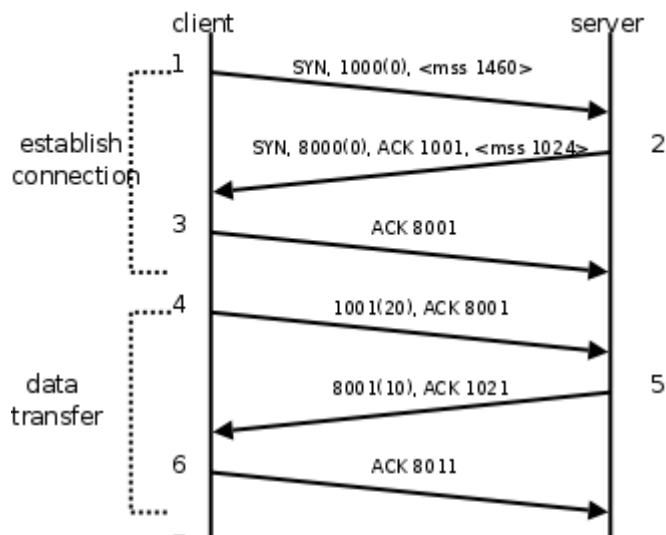
再议TCP协议

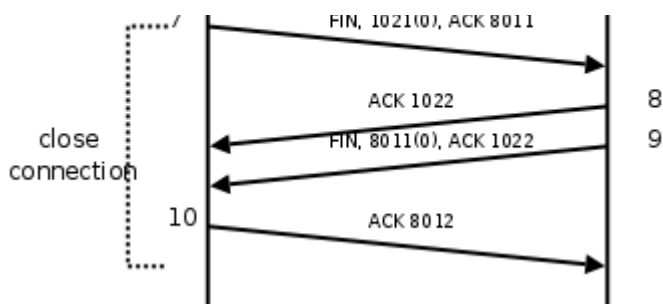
1、TCP数据报，称为流式协议。（传输数据是存在顺序的）



2、TCP链接建立协议：三次握手。

- 1、客户端发送SYN位表示连接请求，mss接收的最大单元1460。
- 2、服务器发送带有SYN位，同时值ACK位表示确认，mss接收的最大单元1024。
- 3、客户端发出ACK对服务器的连接请求进行应答。



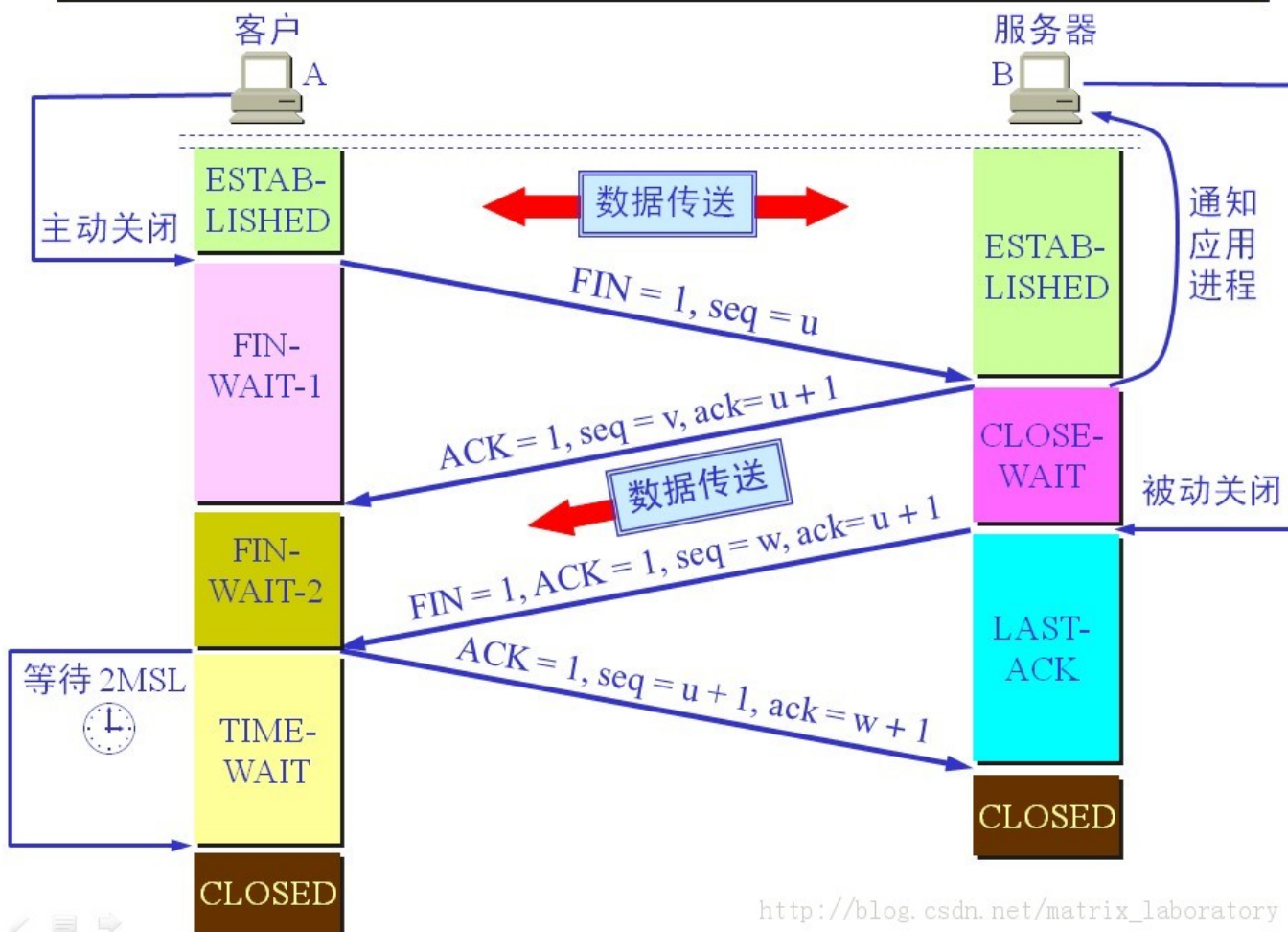


3、TCP终止链接协议：四次握手（内核协议栈处理）

- 1、客户端发送FIN位表示关闭连接的请求。
- 2、服务器发送应答客户端的关闭连接请求。
- 3、服务器再次发送FIN位，向客户端发送关闭连接请求。
- 4、客户端发送应答服务器的关闭连接请求。

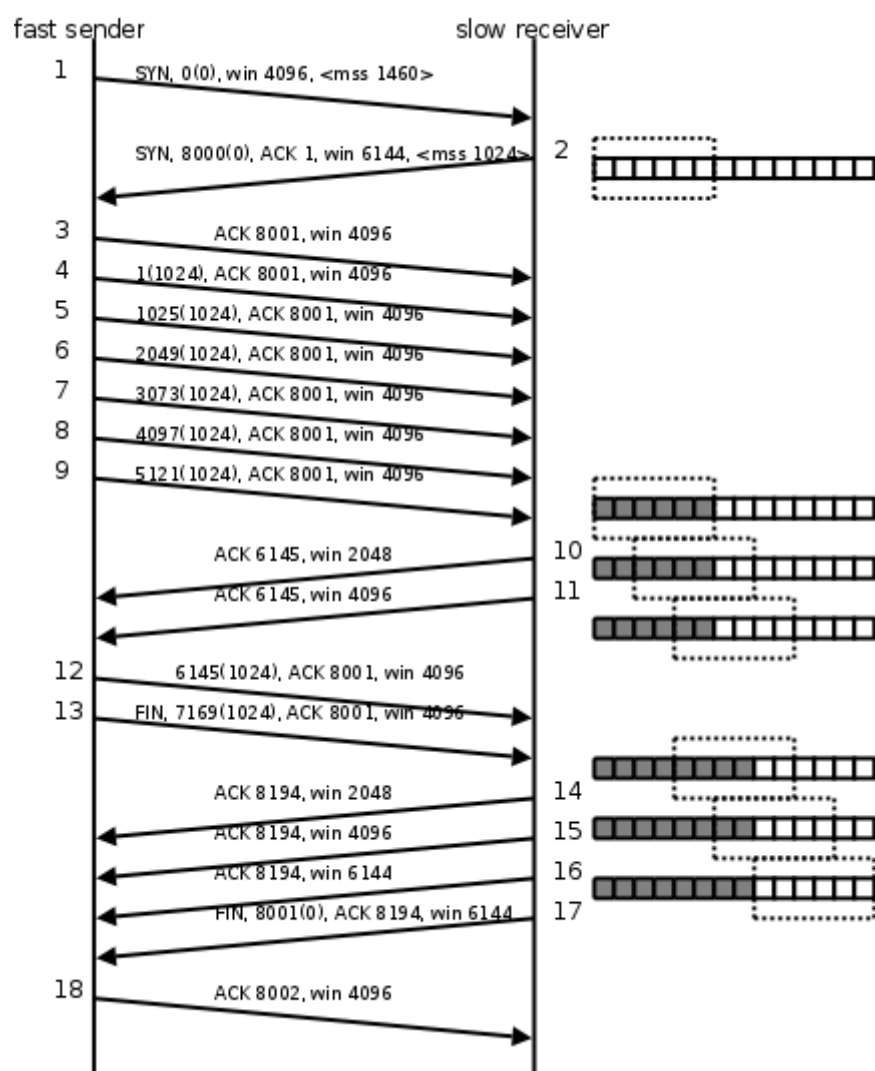
半链接，既是当前客户端断开，服务器还在链接。

TCP 连接必须经过时间 2MSL 后才真正释放掉。



4、TCP流量控制（滑动窗口）：缓冲区

如果发送端发送的速度较快，接收端接收到数据后处理的速度较慢，而接收缓冲区的大小是固定的，就会丢失数据。TCP协议提出“滑动窗口”，也是缓冲区。Win = 6144.

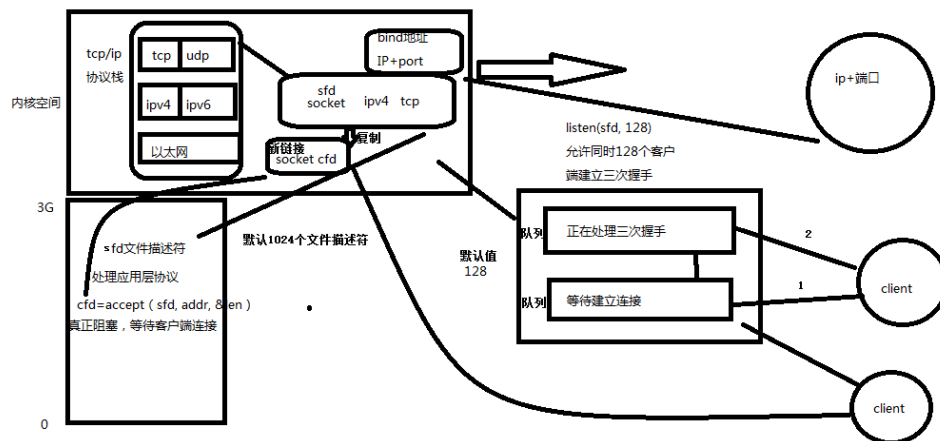


socket套接总结

- 1、tcp/ip协议栈在内核只有一份，每条socket共享这一个协议栈。
- 2、socket打开成功之后，返回一个新的文件描述符。
- 3、listen监听最大128个处理建立连接，超过128个，只能等待。不阻塞
- 4、accept参数能判断出ipv4或是ipv6.切记参数初始化。阻塞，三次握手成功才返回，返回一个新的文件描述符。
- 5、connect参数传入服务器地址和端口号。客户端不需要bind绑定。

6、客户端的源地址端口号是随机分配的，不需要设置

6、客户端的通信端口是随机分配，不需要设置。



Socket api函数出错处理封装

1、wrap.h

```
1 #ifndef __WRAP_H_
2 #define __WRAP_H_
3
4
5 void perr_exit(const char *s);
6 int Accept(int fd, struct sockaddr *sa, socklen_t *salenptr);
7 void Bind(int fd, const struct sockaddr *sa, socklen_t salen);
8 void Connect(int fd, const struct sockaddr *sa, socklen_t salen);
9 void Listen(int fd, int backlog);
10 int Socket(int family, int type, int protocol);
11 ssize_t Read(int fd, void *ptr, size_t nbytes);
12 ssize_t Write(int fd, const void *ptr, size_t nbytes);
13 void Close(int fd);
14 ssize_t Readn(int fd, void *vptr, size_t n);
15 ssize_t Writen(int fd, const void *vptr, size_t n);
16 static ssize_t my_read(int fd, char *ptr);
17 ssize_t Readline(int fd, void *vptr, size_t maxlen);
18
19
20 #endif
```

2. wrap.c

```
1 #include <stdlib.h>
2 #include <errno.h>
3 #include <sys/socket.h>
4
5
6 void perr_exit(const char *s)
7 {
8     perror(s);
9     exit(1);
10 }
11
12
13 int Accept(int fd, struct sockaddr *sa, socklen_t *salenptr)
14 {
15     int n;
16 again:
17     if ( (n = accept(fd, sa, salenptr)) < 0) {
18         if ((errno == ECONNABORTED) || (errno == EINTR))
19             goto again;
20         else
21             perr_exit("accept error");
22     }
23     return n;
24 }
25
26
27 void Bind(int fd, const struct sockaddr *sa, socklen_t salen)
28 {
29     if (bind(fd, sa, salen) < 0)
30         perr_exit("bind error");
31 }
32
33
34 void Connect(int fd, const struct sockaddr *sa, socklen_t salen)
35 {
36     if (connect(fd, sa, salen) < 0)
37         perr_exit("connect error");
38 }
39
40
```

```

41 void Listen(int fd, int backlog)
42 {
43     if (listen(fd, backlog) < 0)
44         perr_exit("listen error");
45 }
46
47
48 int Socket(int family, int type, int protocol)
49 {
50     int n;
51     if ( (n = socket(family, type, protocol)) < 0)
52         perr_exit("socket error");
53     return n;
54 }
55
56
57 ssize_t Read(int fd, void *ptr, size_t nbytes)
58 {
59     ssize_t n;
60 again:
61     if ( (n = read(fd, ptr, nbytes)) == -1) {
62         if (errno == EINTR)
63             goto again;
64         else
65             return -1;
66     }
67     return n;
68 }
69
70
71 ssize_t Write(int fd, const void *ptr, size_t nbytes)
72 {
73     ssize_t n;
74 again:
75     if ( (n = write(fd, ptr, nbytes)) == -1) {
76         if (errno == EINTR)
77             goto again;
78         else
79             return -1;
80     }
81     return n;
82 }
83

```

```

84
85 void Close(int fd)
86 {
87     if (close(fd) == -1)
88         perr_exit("close error");
89 }
90
91
92 ssize_t Readn(int fd, void *vptr, size_t n)
93 {
94     size_t nleft;
95     ssize_t nread;
96     char *ptr;
97     ptr = vptr;
98     nleft = n;
99     while (nleft > 0) {
100         if ( (nread = read(fd, ptr, nleft)) < 0) {
101             if (errno == EINTR)
102                 nread = 0;
103             else
104                 return -1;
105         } else if (nread == 0)
106             break;
107         nleft -= nread;
108         ptr += nread;
109     }
110     return n - nleft;
111 }
112
113
114 ssize_t Writen(int fd, const void *vptr, size_t n)
115 {
116     size_t nleft;
117     ssize_t nwritten;
118     const char *ptr;
119     ptr = vptr;
120     nleft = n;
121     while (nleft > 0) {
122         if ( (nwritten = write(fd, ptr, nleft)) <= 0) {
123             if (nwritten < 0 && errno == EINTR)
124                 nwritten = 0;
125             else
126                 return -1;
127         }

```

```

128     nleft -= nwritten;
129     ptr += nwritten;
130 }
131 return n;
132 }
133
134
135 static ssize_t my_read(int fd, char *ptr)
136 {
137     static int read_cnt;
138     static char *read_ptr;
139     static char read_buf[100];
140     if (read_cnt <= 0) {
141 again:
142         if ( (read_cnt = read(fd, read_buf, sizeof(read_buf))) < 0) {
143             if (errno == EINTR)
144                 goto again;
145             return -1;
146         } else if (read_cnt == 0)
147             return 0;
148         read_ptr = read_buf;
149     }
150     read_cnt--;
151     *ptr = *read_ptr++;
152     return 1;
153 }
154
155
156 ssize_t Readline(int fd, void *vptr, size_t maxlen)
157 {
158     ssize_t n, rc;
159     char c, *ptr;
160     ptr = vptr;
161     for (n = 1; n < maxlen; n++) {
162         if ( (rc = my_read(fd, &c)) == 1) {
163             *ptr++ = c;
164             if (c == '\n')
165                 break;
166         } else if (rc == 0) {
167             *ptr = 0;
168             return n - 1;
169         } else
170             return -1;
171     }

```



```
171     }  
172     *ptr = 0;  
173     return n;  
174 }
```

多进程并发服务器

1、使用多进程并发服务器时要考虑以下几点：

- 1.1父进程最大文件描述个数（父进程中需要close关闭accept返回的新文件描述符）；
- 1.2系统内创建进程个数(内存大小相关);
- 1.3进程创建过多是否降低整体服务性能（进程调度）；

2、服务器端

2.1、server.c

```
1 #include <stdio.h>  
2 #include <stdlib.h>  
3 #include <unistd.h>  
4 #include <string.h>  
5 #include <sys/types.h>  
6 #include <sys/socket.h>  
7 #include <arpa/inet.h>  
8 #include <ctype.h>  
9 #include <signal.h>  
10 #include "wrap.h"  
11  
12  
13 #define SERV_IP "192.168.152.128"  
14 #define SERV_PORT 8090  
15 #define MAXLINE 4096  
16  
17  
18 //捕捉函数
```

```

19 void do_sigchild(int num)
20 {
21     pid_t cpid;
22     while((cpid = waitpid(0,NULL,WNOHANG)) > 0)
23     {
24 #ifdef DEBUG
25         printf("child %d\n is over\n",cpid);
26 #else
27         ;
28 #endif
29     }
30 }
31
32
33 int main(void)
34 {
35     int sfd,cfd,len,i;
36     char buf[MAXLINE],clie_ip[128];
37     pid_t pid;
38     struct sigaction act;
39
40
41     struct sockaddr_in serv_addr,clie_addr;
42     socklen_t clie_len;
43
44
45     sfd = Socket(AF_INET,SOCK_STREAM,0);
46     //构造地址结构
47
48     bzero(&serv_addr,sizeof(serv_addr));
49     serv_addr.sin_family = AF_INET;
50     inet_pton(AF_INET,SERV_IP,&serv_addr.sin_addr.s_addr);
51     serv_addr.sin_port = htons(SERV_PORT);
52
53
54     Bind(sfd,(struct sockaddr *)&serv_addr,sizeof(serv_addr));
55
56
57     Listen(sfd,128);
58     //信号捕捉设置
59     act.sa_handler = do_sigchild;
60     sigemptyset(&act.sa_mask);
61     act.sa_flags = 0;
62

```

```
sigaction(SIGCHLD,&act,NULL);
```

```
while(1)
```

```
{
```

```
    clie_len = sizeof(clie_addr);
```

```
    cfd = Accept(sfd,(struct sockaddr *)&clie_addr,&clie_len);
```

```
    printf("receive client %s\t%d connect\n",inet_ntop(AF_INET,&clie_addr.sin_addr
```

```
    pid = fork();
```

```
    if(pid == 0)
```

```
    {
```

```
        //child
```

```
        Close(sfd);
```

```
        while(1)
```

```
        {
```

```
            len = Read(cfd,buf,sizeof(buf));//阻塞
```

```
            if(len == 0)
```

```
            {
```

```
                printf("client %s is close connect\n",inet_ntop(AF_INET,&clie_ad
```

```
                Close(cfd);
```

```
                exit(1);
```

```
            }
```

```
            for(i = 0;i < len; i++)
```

```
                buf[i] = toupper(buf[i]);
```

```
            Write(cfd,buf,len);
```

```
        }
```

```
    }
```

```
    else if(pid > 0)
```

```
    {
```

```
        //parent
```

```
        Close(cfd);//父进程不干活，子进程负责通信
```

```
    }
```

```
    else
```

```
    {
```

```
        perr_exit("fork err");
```

```
    }
```

```
106     }
107     Close(sfd);
108
109
110     return 0;
111 }
```

3、客户端

3.1 client.c

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <strings.h>
4  #include <string.h>
5  #include <sys/types.h>
6  #include <sys/socket.h>
7  #include <arpa/inet.h>
8  #include "wrap.h"
9
10
11 #define SERV_PROT 8090
12 #define SERV_IP "192.168.152.128"
13
14
15 int main(void)
16 {
17     int cfd, len;
18     struct sockaddr_in saddr;
19     char buf[4096] = "helloworld";
20
21
22     cfd = Socket(AF_INET, SOCK_STREAM, 0);
23
24
25     bzero(&saddr, sizeof(saddr));
26     saddr.sin_family = AF_INET;
```

```

27     inet_pton(AF_INET, SERV_IP, (void*)&saddr.sin_addr.s_addr); // 目标器的ip地址
28     saddr.sin_port = htons(SERV_PROT);
29
30
31     Connect(cfd, (struct sockaddr *)&saddr, sizeof(saddr));
32
33
34     while(fgets(buf, sizeof(buf), stdin) != NULL)
35     {
36         Write(cfd, buf, strlen(buf));
37         len = read(cfd, buf, sizeof(buf));
38         Write(STDOUT_FILENO, buf, len);
39     }
40
41
42     Close(cfd);
43
44
45     return 0;
46 }

```

多线程并发服务器

1、在使用线程模型开发服务器时需考虑以下问题：

- 1、调用进程内最大文件描述符上限；
- 2、线程如有共享数据，考虑线程同步；
- 3、服务于客户端线程退出时，退出处理；（退出值，分离态）
- 4、系统负载，随着链接客户端增加，导致其他线程不能及时得到CPU。

2、服务器

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <string.h>

```

```

5 #include <sys/types.h>
6 #include <sys/socket.h>
7 #include <arpa/inet.h>
8 #include <ctype.h>
9 #include <pthread.h>
10 #include "wrap.h"
11
12
13 #define SERV_IP "192.168.152.128"
14 #define SERV_PORT 8090
15 #define MAXLINE 4096
16
17
18 struct sock_task
19 {
20     int cfd;
21     struct sockaddr_in clie_addr;
22 };
23
24
25
26
27 void *do_task(void *arg)
28 {
29     //处理客户端数据
30     struct sock_task *task = (struct sock_task *)arg;
31     int len,i;
32     char buf[MAXLINE],clie_ip[128];
33
34
35     pthread_detach(pthread_self());
36
37
38     while(1)
39     {
40 again:
41         len = Read(task->cfd,buf,sizeof(buf));
42         if(len == 0)
43         {
44             printf("client %s is close\n",inet_ntop(AF_INET,&task->clie_addr.sin_addr,
45             Close(task->cfd);
46             pthread_exit(0);
47         }
48     }

```

```

48     else if(len < 0)
49     {
50         goto again;
51     }
52
53
54     for(i = 0; i < len; i++)
55         buf[i] = toupper(buf[i]);
56
57
58     Write(task->cfd,buf,len);
59 }
60 }
61
62
63 int main(void)
64 {
65     int sfd,cfd,len,i = 0;
66     char buf[MAXLINE],clie_ip[128];
67
68
69     struct sockaddr_in serv_addr,clie_addr;
70     socklen_t clie_len;
71     pthread_t tid;
72     struct sock_task task[1024];
73
74
75     sfd = Socket(AF_INET,SOCK_STREAM,0);
76
77
78     bzero(&serv_addr,sizeof(serv_addr));
79     serv_addr.sin_family = AF_INET;
80     inet_pton(AF_INET,SERV_IP,&serv_addr.sin_addr.s_addr);
81     serv_addr.sin_port = htons(SERV_PORT);
82
83
84     Bind(sfd,(struct sockaddr *)&serv_addr,sizeof(serv_addr));
85
86
87     Listen(sfd,128); //不阻塞
88
89
90     while(1)
91     {

```

```

92     clie_len = sizeof(clie_addr);
93     cfd = Accept(sfd, (struct sockaddr *)&clie_addr, &clie_len);
94     printf("receive client %s\t%d connect\n", inet_ntop(AF_INET, &clie_addr.sin_addr,
95
96
97     task[i].cfd = cfd;
98     task[i].clie_addr = clie_addr;
99     pthread_create(&tid, NULL, do_task, (void *)&task[i]);
100    i++;
101    }
102
103
104    Close(sfd);
105
106
107    return 0;
108 }

```

3、客户端

```

1  #include <stdio.h>
2  #include <unistd.h>
3  #include <strings.h>
4  #include <string.h>
5  #include <sys/types.h>
6  #include <sys/socket.h>
7  #include <arpa/inet.h>
8  #include "wrap.h"
9
10
11 #define SERV_PROT 8090
12 #define SERV_IP "192.168.152.128"
13
14
15 int main(void)
16 {
17     int cfd, len;
18     struct sockaddr_in saddr;
19     char buff[4096] = "hello world";

```



```
19 char buf[4096] = "hello world";
20
21
22 cfd = Socket(AF_INET, SOCK_STREAM, 0);
23
24
25 bzero(&saddr, sizeof(saddr));
26 saddr.sin_family = AF_INET;
27 inet_pton(AF_INET, SERV_IP, (void*)&saddr.sin_addr.s_addr); // 目标器的ip地址
28 saddr.sin_port = htons(SERV_PROT);
29
30
31 Connect(cfd, (struct sockaddr*)&saddr, sizeof(saddr));
32
33
34 while(fgets(buf, sizeof(buf), stdin) != NULL)
35 {
36     Write(cfd, buf, strlen(buf));
37     len = read(cfd, buf, sizeof(buf));
38     Write(STDOUT_FILENO, buf, len);
39 }
40
41
42 Close(cfd);
43
44
45 return 0;
46 }
```